LLARS - Legacy Script Creator Guide

Traditional AutoHotkey v1 Methods Inside LLARS

This document is for creators who intentionally want to write LLARS scripts using more traditional AutoHotkey v1 methods and syntax.

Legacy here does not mean forbidden. It means lower-level: the script takes responsibility for more configuration reads, randomization, scheduling, focus checks, cleanup, and diagnostics that the current Creator API normally handles.

For new scripts, start with the current Script Creator & Framework Guide. Use this guide when maintaining old scripts, building unusual custom behavior, or when direct AHK syntax is clearer for a genuinely low-level task.

Table of Contents

1. What This Guide Is For
2. What Still Must Stay Framework-Owned
3. Traditional RunCount Lifecycle
4. Traditional First-Run Logic
5. IniRead, Random, and Manual Values
6. Traditional Sleeps and Runtime Estimation
7. Raw Keyboard Input
8. Raw Mouse Movement and Clicking
9. NaturalClick with Resolved Coordinates
10. Raw PixelGetColor
11. Traditional SetTimer Labels
12. Managed Label Timer Compatibility
13. Timed Script Lifecycle
14. Config.ini Metadata Still Matters
15. RuneScape Focus Responsibilities
16. Developer Mode Limitations with Raw AHK
17. Pause, Exit, and Run-State Guards
18. Legacy Helper Reference
19. When Legacy Code Is Reasonable
20. When to Migrate to the Creator API
21. Complete Traditional RunCount Example
22. The Simple Version

1. What This Guide Is For

The current LLARS project is API-first, but AutoHotkey remains available underneath it. Some creators may prefer labels, IniRead, Random, raw SetTimer, or custom MouseMove/PixelGetColor logic. Those patterns can coexist with LLARS as long as the script preserves the framework boundaries and takes responsibility for the behavior the API would normally provide.

> **Legacy note:** Do not use this guide as the default starting point for ordinary new scripts. The current Creator API is shorter, safer around focus, easier to diagnose, and is what the maintained templates are designed around.

2. What Still Must Stay Framework-Owned

- Keep LLARS_Initialize(), the framework-presence check, and the bootstrap include tail from a maintained template.
- Keep exactly one SCRIPT_EDIT_BEGIN marker and one SCRIPT_EDIT_END marker.
- Let the shared LLARS GUI, control hotkeys, configuration validation, and global run state remain intact.
- Use the shared RunCount, Timer, or UntilDone lifecycle; do not invent a separate LLARS run state.
- Use LLARS_WaitForRuneScape() before raw game-facing actions so another application cannot receive input.
- Stop script-owned raw timers during completion/Exit paths.

3. Traditional RunCount Lifecycle

The manual lifecycle remains available for older scripts and low-level creator code:

```
Start:
if (!LLARS_StartRun())
    return

Loop, %runcount%
{
    if (!LLARS_RUNNING)
        break

    LLARS_BeginLoop()

    ; TRADITIONAL SCRIPT CODE

    LLARS_EndLoop()
}

if (LLARS_RUNNING)
    LLARS_RunComplete()
return
```

In this pattern, the creator is responsible for placing BeginLoop/EndLoop correctly and making sure all early exits, errors, and special paths still leave the framework in a valid state.

> **Note:** The current LLARS_RunCount(Func("Run")) wrapper exists specifically so new scripts do not need this boilerplate.

4. Traditional First-Run Logic

```
if (firstrun = 0)
{
    ; FIRST RUN ONLY
```

```
    firstrun := 1
}
else
{
    ; SECOND RUN AND LATER
}
```

LLARS_ResetRunState() initializes the legacy first-run globals before a new RunCount session. This pattern remains understood by the runtime parser for migration compatibility, but current code should prefer ctx.IsFirst.

5. IniRead, Random, and Manual Values

Read a configured hotkey

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
```

Read an area and choose a point

```
IniRead, xmin, Config.ini, Action Location, xmin
IniRead, xmax, Config.ini, Action Location, xmax
IniRead, ymin, Config.ini, Action Location, ymin
IniRead, ymax, Config.ini, Action Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
```

Read a timer range

```
IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
```

Traditional reads give you full control, but they also bypass the typed validation/error behavior of LLARS_ConfigRead* helpers. Validate values before using them when incorrect configuration could produce unsafe behavior.

6. Traditional Sleeps and Runtime Estimation

A raw Sleep works, but RunCount estimation cannot automatically understand arbitrary custom timing logic. If an older script already randomizes a duration and wants the LLARS estimate to track it, the compatibility helpers remain available.

Ordinary estimated wait

```
IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
LLARS_EstimatedSleep(SleepAmount)
```

Final exact wait

```
IniRead, sa1, Config.ini, Final Sleep, min
IniRead, sa2, Config.ini, Final Sleep, max
Random, SleepAmount, %sa1%, %sa2%
LLARS_FinalSleep(SleepAmount)
```

The pre-run parser recognizes only specific standardized/legacy layouts inside the editing markers. Keep min/max reads close to the sleep call and use simple literal section names if you expect the estimator to model the timing.

> **Legacy note:** Raw Sleep by itself is invisible to the standardized RunCount estimate. If accurate pre-run timing matters, use the current LLARS_Sleep() API whenever possible.

7. Raw Keyboard Input

Raw Send bypasses LLARS input transport, including AHI when active, and shared diagnostics; protect focus explicitly:

```
if !LLARS_WaitForRuneScape("Legacy Space key")
    return

Send, {Space}
```

Configured key

```
IniRead, hk, Config.ini, Action Hotkey, hotkey

if !LLARS_WaitForRuneScape("Legacy configured hotkey")
    return

Send, {%hk%}
```

Purposeful hardcoded keys are valid in low-level code. The concern is not that a key is hardcoded; the concern is making sure the intended RuneScape window is foreground immediately before input.

8. Raw Mouse Movement and Clicking

Raw MouseMove/Click bypasses LLARS transport, including AHI when active; prefer Creator API input and diagnostics.

```
if !LLARS_WaitForRuneScape("Legacy MouseMove")
    return

MouseMove, %x%, %y%, %speed%
```

If the code performs more work before a later Click, re-check focus immediately before the click:

```
if !LLARS_WaitForRuneScape("Legacy Click")
    return

Click
```

> **Legacy note:** Do not assume focus is still safe just because it was checked before a long calculation or wait. The user can change foreground windows in the gap.

9. NaturalClick with Resolved Coordinates

```
NaturalClick(x, y)
```

```
NaturalClick(x, y, "right")
```

Direct NaturalClick is the preferred low-level click when your script has already calculated the coordinates. It retains LLARS natural movement and click safety instead of using a raw Click command.

- NaturalClick caps each path at 3000 ms; retries remain unlimited, so user displacement can extend elapsed time.
- It does not click until it reaches the intended pixel.
- Focus changes during movement cause the path to restart only after RuneScape is restored.
- When Developer Mode is open and a matching configured range can be identified, the overlay stays visible during the click and briefly after a successful click.

10. Raw PixelGetColor

Raw pixel reads must not sample another application. Focus gate immediately before PixelGetColor:

```
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y
IniRead, targetColor, Config.ini, Target Color, color

if !LLARS_WaitForRuneScape("Legacy pixel read")
    return

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor = targetColor)
{
    ; handle match
}
```

> **Note:** For normal current code, LLARS_PixelColor()/LLARS_PixelMatches()/LLARS_PixelMatchesAny() are easier and include validation/diagnostics.

11. Traditional SetTimer Labels

Raw SetTimer is still useful when a script intentionally needs label semantics or a fixed polling cadence.

```
interval := LLARS_TimerInterval("Action Timer")
if (interval > 0)
    SetTimer, ActionTimer, -%interval%
return

ActionTimer:
if (!LLARS_RUNNING)
    return

if !LLARS_WaitForRuneScape("Legacy timer action")
    return

; action

interval := LLARS_TimerInterval("Action Timer")
if (interval > 0)
    SetTimer, ActionTimer, -%interval%
return
```

When you own raw timers, you also own rescheduling, stopping them on completion, and preventing late actions after the run ends.

12. Managed Label Timer Compatibility

If you want to keep a label body but gain framework tracking/focus-safe dispatch, LLARS_TimerSchedule() remains available:

```
LLARS_TimerSchedule("ActionTimer", "Action Timer", true)

ActionTimer:
; legacy label body
return
```

This is a compatibility bridge, not the new default. A repeating label timer uses the randomized period selected when scheduled. The modern LLARS_TimerRepeat() callback form chooses a fresh randomized interval after every callback.

LLARS_TimerStop("ActionTimer") and LLARS_TimerStopAll() can stop managed label timers.

13. Timed Script Lifecycle

Traditional Timer scripts still start and end through the shared timed-run lifecycle:

```
Start:
if (!LLARS_StartTimerRun())
    return

SetTimer, Countdown, 1000
; schedule script-specific work
return

EndMsg:
SetTimer, Countdown, Off
; stop every raw script timer here
LLARS_TimerStopAll()
LLARS_EndTimerRun()
Logout()
return
```

Maintained Timer templates own the overall countdown. For scripts that end when task logic decides, use LLARS_StartUntilDoneRun()/LLARS_EndUntilDoneRun(); no duration prompt is used.

14. Config.ini Metadata Still Matters

Traditional creator code can read Config.ini manually, but typed metadata still drives the LLARS setup GUI, Configuration Status, Reset Config, and Developer Mode.

```
[Bank]
type=coordinate
xmin=
xmax=
ymin=
ymax=

[Bank Preset]
type=hotkey
hotkey=

[Target Color]
type=color
color=0xFFFFFF
```

Keep comments above sections so users know what to configure. Raw code is not a reason to make setup less clear.

15. RuneScape Focus Responsibilities

The largest behavioral difference between modern API code and raw AHK is focus responsibility.

Operation	Current API	Raw/legacy responsibility
Click	LLARS_Click() reclaims/verifies RuneScape.	Call LLARS_WaitForRuneScape() immediately before raw mouse input.
Keyboard	LLARS_PressKey()/PressHotkey() recheck immediately before Send.	Call LLARS_WaitForRuneScape() immediately before raw Send.
Managed callback timer	Dispatcher reclaims RuneScape before callback.	Raw SetTimer does nothing automatically.
Pixel read	Pixel helpers wait until RuneScape is foreground; they do not activate it.	Call LLARS_WaitForRuneScape() immediately before PixelGetColor.
Sleep	LLARS_Sleep() allows Alt-Tab during the wait, then restores RuneScape.	Raw Sleep never reclaims focus; gate the next raw action.

Pause and Exit stop the framework from continuing action-time focus reclamation. A creator should never defeat those controls with an independent custom focus loop.

16. Developer Mode Limitations with Raw AHK

Developer Mode shows framework-visible information. The more your script bypasses LLARS helpers, the less of its behavior the framework can label automatically.

- Raw Send may not appear with the same configured-section label as LLARS_PressHotkey().
- Raw Click/MouseMove may not produce the same click target/action events as LLARS_Click()/NaturalClick.
- Raw SetTimer is not automatically represented as a managed timer unless creator code logs it.
- Raw PixelGetColor does not automatically integrate with the same pixel helper diagnostics.

Use Log("EVENT", "Message") for important custom behavior, but avoid turning Developer Mode into a manual execution trace.

17. Pause, Exit, and Run-State Guards

Traditional labels/functions should stop cleanly when LLARS is no longer running.

```
if (!LLARS_RUNNING)
    return
```

For function-style low-level code, LLARS_RunActive() is usually clearer:

```
if !LLARS_RunActive()
    return
```

Do not keep custom action loops running after completion. Raw timers and custom threads need explicit cleanup if the framework does not own them.

18. Legacy Helper Reference

Helper	Why a legacy script may use it
LLARS_StartRun()	Start/validate a manual RunCount lifecycle.
LLARS_BeginLoop()	Start legacy loop bookkeeping and estimator timing.
LLARS_EndLoop()	Record one legacy loop completion.

LLARS_RunComplete()	Perform shared RunCount completion.
LLARS_StartTimerRun() / LLARS_EndTimerRun()	Shared timed-run start/end.
LLARS_EstimatedSleep(time)	Legacy explicit-duration sleep that keeps predictive estimate behavior.
LLARS_FinalSleep(time)	Legacy explicit-duration final exact countdown.
LLARS_TimerInterval(section)	Read/randomize a configured timer range for custom scheduling.
LLARS_TimerSchedule(label, section, repeat)	Managed compatibility wrapper for label timers.
LLARS_WaitForRuneScape(action)	Required focus gate before custom raw game-facing operations.
NaturalClick(x,y)	Low-level safe natural click for resolved coordinates.
Log(event,message)	Record meaningful custom events.

19. When Legacy Code Is Reasonable

- Maintaining an older script where a full conversion would create unnecessary risk.
- Custom mouse movement or driver-level behavior with no equivalent Creator API helper.
- Specialized polling that deliberately needs a fixed raw SetTimer cadence.
- Complex dynamic data/config calculations where direct IniRead is clearer than wrapping everything.
- Debugging or experimenting with AutoHotkey itself while still using the LLARS GUI/lifecycle.

Even in these cases, mix levels thoughtfully: a legacy script can still use LLARS_Click(), LLARS_PressKey(), or managed timers for the parts they fit.

20. When to Migrate to the Creator API

- The code repeats IniRead + Random + NaturalClick for ordinary configured locations.
- The code repeats IniRead + Random + Sleep for standard waits.
- Raw Send is only pressing a configured hotkey or a fixed normal key.
- A label timer exists only to randomize/reschedule the next action.
- Manual RunCount code exists only because the script predates LLARS_RunCount().
- Focus and cleanup bugs are becoming harder to reason about than the task logic itself.

A good conversion keeps the task sequence and purposeful constants intact while deleting framework boilerplate.

21. Complete Traditional RunCount Example

```
Start:
if (!LLARS_StartRun())
    return

Loop, %runcount%
{
    if (!LLARS_RUNNING)
        break

    LLARS_BeginLoop()

    if (firstrun = 0)
    {
        IniRead, xmin, Config.ini, Bank Prime, xmin
        IniRead, xmax, Config.ini, Bank Prime, xmax
        IniRead, ymin, Config.ini, Bank Prime, ymin
        IniRead, ymax, Config.ini, Bank Prime, ymax
        Random, x, %xmin%, %xmax%
        Random, y, %ymin%, %ymax%
        NaturalClick(x, y)
        firstrun := 1
```

```
    }
    else
    {
        IniRead, xmin, Config.ini, Bank Main, xmin
        IniRead, xmax, Config.ini, Bank Main, xmax
        IniRead, ymin, Config.ini, Bank Main, ymin
        IniRead, ymax, Config.ini, Bank Main, ymax
        Random, x, %xmin%, %xmax%
        Random, y, %ymin%, %ymax%
        NaturalClick(x, y)
    }

    IniRead, hk, Config.ini, Bank Preset, hotkey
    if !LLARS_WaitForRuneScape("Legacy Bank Preset")
        break
    Send, {%hk%}

    IniRead, sa1, Config.ini, Final Sleep, min
    IniRead, sa2, Config.ini, Final Sleep, max
    Random, SleepAmount, %sa1%, %sa2%
    LLARS_FinalSleep(SleepAmount)

    LLARS_EndLoop()
}

if (LLARS_RUNNING)
    LLARS_RunComplete()
return
```

Legacy note: This example is intentionally verbose so the responsibilities are visible. The modern Creator API version is dramatically shorter and should be preferred for a new script.

22. The Simple Version

- Traditional AHK is allowed, but the script owns more responsibility.
- Keep the LLARS template/bootstrap, editing markers, start/end lifecycle, and shared controls.
- Before raw Send, MouseMove, Click, or PixelGetColor, call LLARS_WaitForRuneScape() immediately before the physical operation.
- Stop raw timers yourself and guard them with LLARS_RUNNING/LLARS_RunActive().
- Use LLARS_EstimatedSleep()/LLARS_FinalSleep() only for explicit legacy durations that need estimator integration.
- Use NaturalClick rather than raw Click when you already have resolved coordinates.
- Mix in current Creator API helpers whenever they simplify a normal action.
- For new ordinary scripts, use the current Creator API documentation instead.